

Decision Tree | Assignment

Question 1: What is a Decision Tree, and how does it work in the context of classification?

Ans - A **Decision Tree** is a supervised machine learning algorithm used for both **classification and regression**, but it is especially popular for classification tasks because of its simple, tree-like structure that is easy to understand and interpret. In the context of classification, a decision tree works by splitting the dataset into smaller subsets based on different conditions on the input features. It starts from a root node, which represents the entire dataset, and then selects the best feature that separates the data most effectively using measures like **Gini Impurity or Information Gain (Entropy)**. Based on this feature, the data is split into branches, and this process continues recursively, forming internal decision nodes until it reaches leaf nodes. Each leaf node represents a final class label or decision. For example, a decision tree can classify whether a person will buy a product or not based on features like age, income, and spending score. The model follows a series of if-else conditions to reach a final prediction, making it highly interpretable. In summary, a decision tree works by repeatedly splitting data based on feature conditions until it reaches a final classification outcome.

Question 2: Explain the concepts of Gini Impurity and Entropy as impurity measures. How do they impact the splits in a Decision Tree?

Ans - **Gini Impurity and Entropy** are two important impurity measures used in **Decision Trees** to decide the best feature for splitting the data at each node in a classification problem. Both measures help the algorithm evaluate how “pure” or “mixed” a node is in terms of class labels, and the goal of a decision tree is to create splits that result in the most pure child nodes.

Gini Impurity measures the probability of incorrectly classifying a randomly chosen element if it were labeled according to the distribution of labels in the node. It ranges from 0 to a maximum value (for binary classification, up to 0.5), where **0 means a perfectly pure node** containing only one class. A lower Gini value indicates a better split because it means the node contains mostly a single class.

Entropy, on the other hand, comes from information theory and measures the level of uncertainty or randomness in the data. It is calculated using the probability distribution of classes in a node. Entropy is 0 when the node is completely pure and increases as the data becomes more mixed.

Both Gini and Entropy are used to decide splits by calculating how much they decrease after a split, known as **Information Gain (for Entropy) or Gini Gain**. The feature that produces the highest reduction in impurity is chosen for splitting. In practice, both measures often produce similar trees, but Gini is slightly faster to compute, while Entropy is more mathematically grounded. Thus, these impurity measures directly control how the decision tree grows by ensuring that each split increases the purity of the resulting nodes.

Question 3: What is the difference between Pre-Pruning and Post-Pruning in Decision Trees? Give one practical advantage of using each.

In **Decision Trees**, overfitting is a common problem where the model becomes too complex and performs well on training data but poorly on unseen data. To avoid this, two main techniques are used: **pre-pruning** and **post-pruning**, which control the growth of the tree in different ways.

Pre-pruning is a technique where the growth of the decision tree is stopped early, before it becomes too complex. This is done by setting conditions such as maximum depth of the tree, minimum number of samples required to split a node, or minimum impurity decrease. In simple terms, the tree is not allowed to grow fully if certain stopping criteria are met. The main practical advantage of pre-pruning is that it reduces computational cost and training time because the tree is smaller and built earlier, and it also helps prevent overfitting at an early stage.

Post-pruning, on the other hand, allows the decision tree to grow fully first and then removes unnecessary branches after the full tree has been created. It works by cutting back parts of the tree that do not contribute significantly to prediction accuracy on validation data. The main practical advantage of post-pruning is that it generally produces a more accurate and reliable model because it considers the full structure of the tree first and then simplifies it based on performance.

Question 4: What is Information Gain in Decision Trees, and why is it important for choosing the best split?

Ans - **Information Gain** is a key concept used in **Decision Trees** to select the best feature for splitting the data at each node. It is based on the idea of **Entropy**, which measures the impurity or randomness in a dataset. Information Gain tells us how much the uncertainty in the data is reduced after splitting it using a particular feature. Mathematically, it is calculated as the difference between the entropy of the parent node and the weighted entropy of the child nodes after the split. A higher Information Gain means that the split has significantly reduced impurity and has made the data more organized or pure.

Information Gain is important because it helps the decision tree algorithm choose the **best possible feature for splitting** at each step. The feature that gives the highest Information Gain is selected as it results in the most effective separation of classes. This ensures that each split improves the quality of classification by making the resulting groups more homogeneous. Without Information Gain, the tree would not have a systematic way to decide which feature to split on, which could lead to poor performance and less accurate predictions. Therefore, Information Gain plays a crucial role in building an efficient and accurate decision tree model by guiding it toward the most informative splits.

Question 5: What are some common real-world applications of Decision Trees, and what are their main advantages and limitations?

Ans - **Decision Trees** are widely used machine learning models because they are simple, interpretable, and effective for both classification and regression problems. In real-world applications, decision trees are commonly used in **banking and finance** for credit risk

analysis and loan approval decisions, where they help determine whether a customer is likely to repay a loan based on factors like income, credit history, and age. They are also used in **healthcare** for disease diagnosis, such as predicting whether a patient has a particular disease based on symptoms and medical test results. In **marketing**, decision trees are used for customer segmentation and predicting customer behavior, such as whether a customer will buy a product or not. They are also applied in **fraud detection**, where transactions are classified as normal or suspicious based on patterns in the data.

The main advantages of decision trees include their **high interpretability**, as the model looks like a simple flowchart of if-else rules that can be easily understood even by non-technical users. They also require little data preprocessing, can handle both numerical and categorical data, and work well with non-linear relationships. However, decision trees also have some limitations. They are prone to **overfitting**, especially when the tree becomes too deep and captures noise in the data. They can also be unstable because small changes in data may lead to a completely different tree structure. Additionally, they may struggle with very complex datasets compared to ensemble methods like Random Forests. Therefore, while decision trees are powerful and easy to use, they often need techniques like pruning or ensemble learning to improve performance.

Question 6: Write a Python program to:

- Load the Iris Dataset
- Train a Decision Tree Classifier using the Gini criterion
- Print the model's accuracy and feature importances

Ans -

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
```

```
iris = load_iris()
X = iris.data
y = iris.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1
)
```

```
model = DecisionTreeClassifier(criterion='gini', random_state=1)
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
importances = model.feature_importances_
```

```
print("Accuracy:", accuracy)
```

```
print("Feature Importances:", importances)
```

Outputs -

Accuracy: 1.0

Feature Importances: [0. 0.01667014 0.90614339 0.07718647]

Question 7: Write a Python program to:

- Load the Iris Dataset
- Train a Decision Tree Classifier with max_depth=3 and compare its accuracy to a fully-grown tree

Ans - from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

```
iris = load_iris()  
X = iris.data  
y = iris.target
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=1  
)  
#model 1  
model_limited = DecisionTreeClassifier(max_depth=3, random_state=1)  
model_limited.fit(X_train, y_train)  
y_pred_limited = model_limited.predict(X_test)  
acc_limited = accuracy_score(y_test, y_pred_limited)  
  
#model 2  
model_full = DecisionTreeClassifier(random_state=42)  
model_full.fit(X_train, y_train)  
y_pred_full = model_full.predict(X_test)  
acc_full = accuracy_score(y_test, y_pred_full)
```

```
print("Accuracy (max_depth=3):", acc_limited)
print("Accuracy (full tree):", acc_full)
```

Outputs-

```
Accuracy (max_depth=3): 1.0
Accuracy (full tree): 1.0
```

Question 8: Write a Python program to:

- Load the Boston Housing Dataset
- Train a Decision Tree Regressor
- Print the Mean Squared Error (MSE) and feature importances

.

Ans -

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error
```

```
data = fetch_california_housing()
```

```
X = data.data
y = data.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1)
```

```
model = DecisionTreeRegressor(random_state=1)
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
mse = mean_squared_error(y_test, y_pred)
```

```
importances = model.feature_importances_
```

```
print("Mean Squared Error:", mse)
print("Feature Importances:", importances)
```

Outputs -

```
Mean Squared Error: 0.495235205629094
Feature Importances: [0.52850909 0.05188354 0.05297497 0.02866046
0.03051568 0.13083768
0.09371656 0.08290203]
```

Question 9: Write a Python program to:

- Load the Iris Dataset
- Tune the Decision Tree's max_depth and min_samples_split using GridSearchCV

- Print the best parameters and the resulting model accuracy

Ans -

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
```

```
iris = load_iris()
X = iris.data
y = iris.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1
)
```

```
dt = DecisionTreeClassifier(random_state=1)
```

```
param_grid = {
    "max_depth": [2, 3, 4, 5, None],
    "min_samples_split": [2, 5, 10]
}
```

```
grid_search = GridSearchCV(
    estimator=dt,
    param_grid=param_grid,
    cv=5
)
```

```
grid_search.fit(X_train, y_train)
```

```
best_model = grid_search.best_estimator_
```

```
y_pred = best_model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Best Parameters:", grid_search.best_params_)
print("Test Accuracy:", accuracy)
```

Outputs -

```
Best Parameters: {'max_depth': 4, 'min_samples_split': 2}
Test Accuracy: 1.0
```

Question 10: Imagine you're working as a data scientist for a healthcare company that wants to predict whether a patient has a certain disease. You have a large dataset with mixed data types and some missing values. Explain the step-by-step process you would follow to:

- Handle the missing values
- Encode the categorical features
- Train a Decision Tree model
- Tune its hyperparameters
- Evaluate its performance And describe what business value this model could provide in the real-world setting.

Ans - In this healthcare prediction problem, the first step is to **handle missing values** by filling numerical features with mean/median and categorical features with mode, ensuring the dataset is complete and usable for modeling. Next, **categorical features are encoded** using Label Encoding or One-Hot Encoding so that all inputs are converted into numerical form, which is required for machine learning algorithms. After preprocessing, a **Decision Tree model is trained** using the training dataset, where it learns patterns between patient features and disease outcomes by creating decision rules based on impurity measures like Gini or Entropy.

Then, the model is **tuned using hyperparameters** such as max_depth, min_samples_split, and min_samples_leaf using GridSearchCV or RandomizedSearchCV to improve performance and reduce overfitting. Finally, the model is **evaluated using metrics** like accuracy, precision, recall, F1-score, and confusion matrix, with recall being especially important in healthcare to avoid missing disease cases. In real-world use, this model provides strong **business value** by enabling early disease detection, supporting doctors in diagnosis, reducing treatment delays, lowering healthcare costs, and improving overall patient care through data-driven decisions.